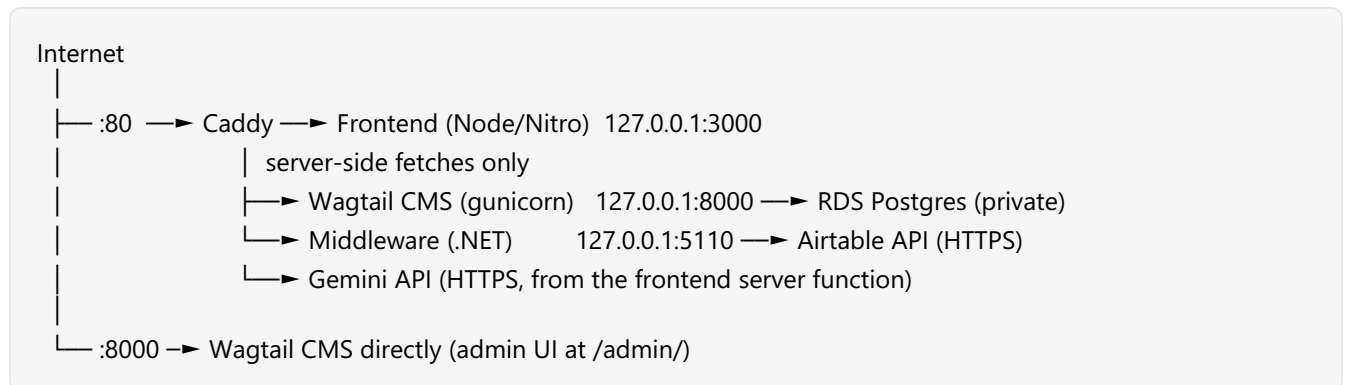


AIME Ceremony-In — AWS Deployment

Single-EC2 deployment of all three services. Live URL: <http://3.217.19.123/> (plain HTTP, no domain/TLS yet).

1. Architecture



All service-to-service calls are loopback and happen server-side (TanStack Start createServerFn), so the browser never talks to the CMS API or middleware directly and no CORS/public exposure is needed between them. Only ports 80 (site) and 8000 (CMS admin) are meant for the browser; the middleware (5110) is loopback-only.

Service	Repo / source	Runtime	Port	systemd unit	Run as
Frontend	AIME-Ceremony-In	Node (Nitro node-server preset)	3000 (loopback)	aime-ceremony-in	aimefrontend
CMS	aime-ceremony-cms	Python 3.14 venv, Django 5.2 + Wagtail 7.4, gunicorn (2 workers)	8000 (public)	aime-ceremony-cms	aimecms
Middleware	aime-mdlwr	Self-contained .NET, installed via its own RPM	5110 (loopback)	aime-mdlwr (shipped by the RPM)	aime
Reverse proxy	—	Caddy	80	caddy	caddy

2. AWS resources (account 723414444862, us-east-1)

Resource	Details
EC2	i-0d406909c59087ea8, t3.small, Fedora 44, AZ us-east-1d, VPC vpc-04c73cf32169089d8
Elastic IP	3.217.19.123 (stable across stop/start)

EC2 security group	sg-05259cf83db5facb5 (aime-mdlwr-ec2-sg): inbound 80, 443, 8000 from anywhere; 22 from a single admin IP only
IAM instance profile	aime-mdlwr-ec2-ssm-profile — AmazonSSMManagedInstanceCore + scoped read of the three Secrets Manager secrets + S3 access to the ceremony-in bucket
RDS	aime-ceremony-cms-db, PostgreSQL 17.11, db.t4g.micro, encrypted, not publicly accessible; SG sg-04dddd3976cea2146 allows 5432 only from the EC2 SG
S3	bucket ceremony-in: cms-media/ (Wagtail uploads via django-storages), deploy-src/ (source tarballs/scripts used to ship code to the box)
Secrets Manager	Gemini API key (aime/gemini-api-key), Airtable token (aime/airtable-token), plus the CMS SECRET_KEY / DATABASE_URL values
SSM	Used for all remote administration — no SSH key needed (see §6)

App Runner was tried first and abandoned; no App Runner services remain. Leftovers that can be deleted if unused: the App Runner GitHub connection aime-github-connection, the VPC connector aime-cms-vpc-connector, and its SG sg-08f81c759d110cf2b.

3. On-box layout

Path	Purpose
/opt/aime-ceremony-in	Frontend source + .output/ build
/opt/aime-ceremony-cms	CMS source, venv/, collected static
/opt/aime-mdlwr	Middleware source (RPM built from here; app installed to /usr/lib/aime-mdlwr)
/etc/aime-ceremony-in/env (600)	NODE_ENV, PORT=3000, WAGTAIL_API_URL=http://127.0.0.1:8000, CEREMONY_API_URL=http://127.0.0.1:5110, GEMINI_API_KEY
/etc/aime-ceremony-cms/env (600)	DJANGO_SETTINGS_MODULE=ceremony_cms.settings.production, SECRET_KEY, DATABASE_URL, ALLOWED_HOSTS=3.217.19.123,127.0.0.1,localhost, CSRF_TRUSTED_ORIGINS, AWS_STORAGE_BUCKET_NAME, AWS_S3_REGION_NAME, DISABLE_HTTPS_REDIRECT=1
/etc/aime-mdlwr/aime-mdlwr.env (640 root:aime)	AIRTABLE_TOKEN, AIRTABLE_BASE_ID, AIRTABLE_TABLE, CORS_ORIGINS, HOST=127.0.0.1, PORT=5110
/etc/caddy/Caddyfile	http://3.217.19.123, :80 { reverse_proxy 127.0.0.1:3000 }
/etc/systemd/system/aime-ceremony-{in,cms}.service	App units (Restart=always)

<code>/etc/systemd/system/aime-ceremony-in.service.d/order.conf</code>	Frontend After=/Wants= CMS
<code>/etc/systemd/system/caddy.service.d/order.conf</code>	Caddy After=/Wants= frontend

Secrets are written to these env files from Secrets Manager at setup time; they are never committed to git.

4. Startup order

All four units are enabled, so they come back after a reboot. Ordering (via systemd drop-ins):

1. aime-ceremony-cms (and aime-mdlwr) — independent
2. aime-ceremony-in — After= + Wants=aime-ceremony-cms.service
3. caddy — After= + Wants=aime-ceremony-in.service

Wants= (not Requires=) is deliberate: the frontend has hardcoded fallbacks for visa paths/agreements, so a CMS outage degrades content but shouldn't take the site down.

5. Configuration notes and gotchas

- ALLOWED_HOSTS must include 127.0.0.1. The frontend calls the CMS at `http://127.0.0.1:8000`; without it Django returns HTTP 400 (DisallowedHost) and the frontend silently falls back to hardcoded content. This was the "CMS not working" bug.
- DISABLE_HTTPS_REDIRECT=1 turns off SECURE_SSL_REDIRECT, secure cookies, and HSTS in `ceremony_cms/settings/production.py`. Remove it once real TLS is in place.
- No HTTPS. Let's Encrypt rejects .amazonaws.com hostnames, and there is no domain. Browsers show "Not Secure"; the CMS admin login and session cookies travel unencrypted. Getting a domain, pointing it at 3.217.19.123, changing the Caddyfile site address to the domain, and dropping DISABLE_HTTPS_REDIRECT gives automatic HTTPS.
- SELinux: the middleware RPM ships no policy, so its process was denied outbound HTTPS to Airtable. A minimal audit2allow-generated module (aime-mdlwr-net) is loaded. A rebuilt instance needs it again.
- RDS access is by security-group reference (EC2 SG → RDS SG on 5432). A replacement instance must be added to the RDS SG.
- Django pinned to 5.2 (Django >=5.2, <5.3); Wagtail 7.4 supports it. Python 3.14 on the box.
- The frontend is built with Nitro's node-server preset (`vite.config.ts`), not the default Cloudflare preset.
- The stray JM X AIME NATION/ folder in the frontend repo is untracked and must never be committed or shipped in a deploy tarball (use `--exclude`).

6. Operating the box

There is no SSH key dependency. Use SSM (IAM permissions required locally):

```
aws ssm send-command --instance-ids i-0d406909c59087ea8 \  
  --document-name AWS-RunShellScript --region us-east-1 \  
  --parameters 'commands=["systemctl is-active aime-ceremony-in aime-ceremony-cms aime-mdlwr caddy"]'  
aws ssm get-command-invocation --command-id <id> --instance-id i-0d406909c59087ea8 --region us-east-1
```

Tips on Windows/Git Bash: prefix with `MSYS_NO_PATHCONV=1` if arguments start with `/`; set `PYTHONUTF8=1` to avoid console-encoding crashes; for multi-line scripts, upload to `s3://ceremony-in/deploy-src/` and download+run on the instance rather than inlining heredocs.

Useful commands on the instance:

```
systemctl status aime-ceremony-in aime-ceremony-cms aime-mdlwr caddy  
journalctl -u aime-ceremony-cms -n 100 --no-pager  
curl -s http://127.0.0.1:5110/health      # middleware -> {"status":"ok"}  
curl -sI http://127.0.0.1:8000/admin/    # CMS -> 302 to login  
curl -sI http://127.0.0.1:3000/         # frontend -> 200
```

Deploying an update

Code is shipped as tarballs via S3, not git-cloned on the box.

1. `tar czf` the repo (exclude `node_modules`, `.git`, `venv`, `.output`, `JM X AIME NATION`) and `aws s3 cp` it to `s3://ceremony-in/deploy-src/`.
2. Via SSM: download to `/opt/<service>`, then
 - Frontend: `npm ci && npm run build && systemctl restart aime-ceremony-in`
 - CMS: `venv/bin/pip install -r requirements.txt && venv/bin/python manage.py migrate --noinput && venv/bin/python manage.py collectstatic --noinput && systemctl restart aime-ceremony-cms`
 - Middleware: rebuild the RPM with `TMPDIR=/var/tmp ./packaging/build-rpm.sh <version-matching-spec>` and `dnf install` it (fix CRLF line endings on `.sh/.spec` first if the tarball came from Windows).
3. Verify with the curl checks above.

7. CMS admin

- URL: `http://3.217.19.123:8000/admin/`, superuser `admin`. The initial password was generated and shared in chat — change it on first login and prefer a personal account.

- Visa Paths and Agreements are edited here (models in ceremony_content). Until content is added, the frontend uses hardcoded fallbacks.

8. Verified behavior

End-to-end test on the live site: Welcome → Path → Identity → Story → Gemini evaluation → Airtable record created (test record deleted afterward). All four units enabled and active.

9. Known limitations / suggested next steps

- Add a domain + HTTPS (highest-value follow-up); then close public port 8000 by proxying the CMS admin through Caddy on 443.
- The RDS instance is single-AZ with default backups; no CloudWatch alarms or log shipping configured.
- Deploys are manual (tarball + SSM). A small script or CI job would remove the friction.
- Branches deploy/aws-app-runner in both repos hold the abandoned App Runner history (apprunner.yaml, S3 storage and Django-version changes). The S3-storage, Django 5.2, and DISABLE_HTTPS_REDIRECT changes are still needed by the EC2 deployment and should be merged to main.